

Architecture Q&A

Team Look Mom I am Quant · START Hack 2026 · UNRISKOMEGA case

Answers are written to be said out loud, and match the code on `main`.

The big picture

Walk me through the architecture in thirty seconds.

Three layers. A **facts engine** reads your client export and reference data and computes every sentence the product may say, each with its source: fund look-through, target allocations, suitability, proposals, market impact. A **reasoning layer** links those facts into a graph and answers questions by choosing facts, or paths through the graph; it never writes new text. A **delivery layer** is one Python API with a push socket serving the phone app and the dashboard, plus the live listener that turns call audio into questions. Everything expensive is precomputed, so a call is a lookup, not a computation.

Why precompute instead of generating on demand?

Because the moment that matters is the ring. Every client's briefing is ready before the call and recomputed when the market moves or a file is uploaded. At ring time the server returns a finished briefing: 0.23 seconds measured on the API. A model-generated briefing takes six to eight seconds, which is why phrasing runs in the background.

Is there a database?

Not in the prototype. Client data is loaded into memory from your export format; caches are files (profiles, translations, news snapshots, recorded runs). In production the store reads from URO's database or API and the caches move to a shared store. The engine doesn't change: a client object in, facts out.

Why not RAG with a vector database?

Retrieval over text is the wrong tool for numbers. A vector search finds a paragraph that mentions tech; it cannot tell you tech is 9.6% of this client's book after breaking down eight funds. We compute facts deterministically and use the model only to choose between them. The one place we retrieve text is news and bank house views, and there we only quote sentences found word for word in the source.

The facts engine

How do you get exposure through funds?

From your fund look-through table, about 48,000 rows: each fund splits into industry, currency, region and asset class, and each fund's rows sum to 100%. We weight the client's holding by those rows. Direct holdings use the security's own classification. What can't be classified is reported as not classified, never guessed.

Your data had quirks. What did you handle?

Missing values that are sometimes absent and sometimes null; a "Consolidated" portfolio that double counts one client (skipped, so totals match the client's assets); industry names that differ between tables (mapped to one set); German display names (translated); and real account numbers in the export, which are never output, with a test for it.

How do you compare holdings with the target allocation?

In the same coarse categories the targets are set in; the securities table has dedicated fields for this. A class outside its minimum-maximum band becomes a health fact and a rebalance action, weighted by how far outside the band it is.

The market model

Where does market data come from?

Two paths, one format. Stage scenarios are simulated, in Bloomberg-shaped ticks (ticker, field, value), and labelled simulated in every source. We also snapshot real daily moves from Yahoo Finance, with sector funds standing in for sector indices. A licensed Bloomberg or SIX feed plugs into the same function; we have not connected one.

How does a market move become a client's loss?

Per holding: the equity part through its industry move, a currency move unless it is a franc-hedged share class, and separate bond, gold and crypto moves. Per-holding impacts add up exactly to the portfolio total, which is what makes the "where did the money go" breakdown exact.

Weaknesses of that model?

Sector moves stand in for single-stock prices, and without bond duration we never claim a specific bond's price move. Holdings nothing maps to are counted and shown as not included. With a per-security price feed the same code becomes exact.

How do you know a fund is hedged?

From the share-class name ("CHF Hedged"), read from the full security name before it is shortened for display.

Reasoning and answers

How does the system answer a question during a call?

Small talk is filtered first. Then, first match wins: a **chain of facts** for why / where did it go / versus the market / did the changes; the **question type** (whole-portfolio overview, open, advice, changes, withdrawal); **Apertus choosing facts by number** (it can only return numbers, never text); **keywords** as a last resort. If the question names a subject and no chosen fact mentions it, the answer is dropped: an empty answer beats a wrong one. Every reply is then rewritten into words the advisor can say, numbers untouched.

What is a chain?

A path through a graph the code builds per client and market. Nodes are facts; links are dependencies the code computed (holding in an industry, industries adding up to the total, a hedge removing the dollar move, a proposal buying a holding). Five chains are built by code; for other why-questions the model may propose a path, rejected unless every step is linked to the next. Sentences are the facts' own text.

Can the "because" be wrong?

It only appears where the code knows the dependency, and arithmetic comes from code, so "everything else" is the computed remainder and a breakdown adds up to the franc. It cannot explain causes outside the data: "why did chip stocks fall" gets the headline and stops.

Where exactly does the LLM run?

Four places: rephrasing the dashboard briefing in the background (a checker rejects added numbers, causes, severity words or changed quotes); choosing fact numbers during a call; proposing chain paths (validated against the graph); reading client notes once into a profile (quotes must match the notes word for word). It never computes a number, and nothing it writes reaches the screen unchecked.

Which model, and why?

Apertus 70B, the Swiss open model, hosted by Swisscom, through a standard OpenAI-style interface, with OpenAI as an optional fallback behind the same interface. Text about clients stays in Switzerland, and choosing facts takes about 0.4 seconds.

The live call

How does live listening work?

Call audio streams to ElevenLabs real-time speech-to-text in 100 ms chunks, with end of sentence tuned to half a second of silence. Each finished sentence is sorted: small talk, instruction, request, question, information, other. Questions go to the answer engine; the rest become quiet lines for the call note. The card lands 0.6 to 1.1 seconds after the client stops talking.

How would you get a real phone call's audio?

Not from the phone app: iOS and Android do not allow it. In production it comes from the bank's advisory phone system, which already records calls. The listener does not care where audio comes from; in the prototype it is fed by the phone's microphone or the recorded demo call.

What happens when someone gives an instruction?

It is never executed. "Sell the Novartis" is recorded as an instruction under "confirm in writing" in the call note, and the follow-up email says nothing has been executed and a proposal follows for written confirmation.

Reliability

What happens when a dependency fails?

Apertus down: plain fact sentences and keyword answers. Voice down: the cards still work. No email settings: a local outbox. Network down: offline replay of a recorded call. A malformed uploaded client is rejected on its own.

Concurrency?

One async server; slow work (model, voice) runs in thread pools; background phrasing uses four workers and skips clients in progress; starting a new demo call stops the old one first.

Testing?

177 offline tests plus live tests on demand: every client under every scenario (sources present, no German, no account numbers), breakdowns adding up, model output restricted to the list, the listener end to end, email and demo flows. A file of realistic call sentences with expected outcomes, extended with what the judges say; every live call is logged sentence by sentence for review.

Scale, security, production

Does it scale?

Per client it is milliseconds of Python, so thousands of clients per market event is seconds. The paid parts (model, speech) run for background phrasing, per question, and per minute of live audio, so cost follows call time, not book size.

Security?

The prototype has no login; production sits behind the bank's single sign-on and device management. Keys live in environment files, account numbers are never output, and call logs stay out of version control and would follow the bank's recording retention rules.

Does client data leave Switzerland?

Text does not (Apertus on Swisscom). Voice does in the prototype, because ElevenLabs processes the audio; for production, speech moves to a Swiss-hosted or on-premise model behind the same interface.

How long to production?

Integration, not invention: read from URO instead of the export, a licensed price feed, audio from the advisory phone system, single sign-on, Swiss-hosted speech. A read-only pilot with three advisors needs only the first two.

What would you build next?

Measure the "probably calling about" guesses against the 206 consultation reasons your advisors logged; a per-security price feed; Swiss-hosted speech; short memory so "why did *that* happen?" knows what "that" refers to.